🏛️ LAKEHOUSE TABLE FORMATS: DELTA VS ICEBERG

Production Reference: Transaction Logs, Concurrency Control, Deletion Vectors & Physical File Layouts

DELTA LAKE 3.2+ | APACHE ICEBERG 1.5+ | UNIFORM INTEROP | ISO 216 A4

📜 Transaction Log Architecture: Delta vs Iceberg

Storage Engine

```
-- DELTA LAKE LOG STRUCTURE (_delta_log/)
table_root/
  ├── _delta_log/
  │   ├── 00000000000000000000.json      -- Atomic commit record
  │   ├── 00000000000000000001.json
  │   └── 00000000000000000010.checkpoint.parquet -- Compacted state every 10 commits
  └── part-00000-xxxx.parquet

-- APACHE ICEBERG METADATA TREE (metadata/)
table_root/
  ├── metadata/
  │   ├── v1.metadata.json               -- Table schema, partitions, snapshots
  │   ├── snap-102938475.avro            -- Manifest list: links all active manifests
  │   └── manifest-uuid-1.avro           -- Manifest file: individual data file stats
  └── data/
      └── part-00000-yyyy.parquet
```

🔍 LOG MECHANICS COMPARISON

- **Delta Lake:** Linear JSON chain with periodic Parquet checkpoints; linear replay reconstructed forward.
- **Apache Iceberg:** Hierarchical directed acyclic graph (DAG) of Avro manifests; permits zero-scan partition pruning.

🔒 Concurrency Control & Snapshot Isolation

ACID Transactions

```
-- Optimistic Concurrency Control (OCC) Collision Rules
1. Delta Lake OCC:
   - Reads current table state version V.
   - Attempts to write commit file V+1.json.
   - If collision occurs, executes mutual conflict resolution:
     ↳ Checks if concurrent transaction modified disjoint files/partitions.
     ↳ If disjoint, re-bases and commits as V+2.json without failing!
2. Apache Iceberg OCC:
   - Validates snapshot S_current at catalog root.
   - Commits new snapshot S_new via atomic compare-and-swap (CAS).
   - Catalogs: REST Catalog, AWS Glue, DynamoDB, JDBC, or Hive Metastore.
3. Cloud Store Mutex Requirements:
   - S3 requires catalog lock or DynamoDB lock-manager (due to lack of put-if-absent).
   - Azure ADLS Gen2 & Google Cloud Storage support native conditional writes.
```

🔍 CONCURRENCY INVARIANTS

- **Write Conflict Rule:** Concurrent `UPDATE` or `MERGE` on overlapping partitions fails with `ConcurrentModificationException`.
- **Concurrent Append:** Multiple concurrent append jobs commit seamlessly without retry in both formats.

🗑️ Deletion Mechanics: COW vs MOR vs Vectors

Row Mutation

```
-- Row-Level Modification Mechanics
1. Copy-On-Write (COW):
   - Rewrites entire Parquet file containing modified/deleted row.
   - High write amplification, maximum read performance.
2. Iceberg Merge-On-Read (MOR):
   - Positional Deletes: Avro file stores (data_file_path, row_position).
   - Equality Deletes: Stores equality predicate (e.g. id = 100).
   - Read penalty: Engine must join data files with delete files on scan.
3. Delta Lake Deletion Vectors (DVs):
   - Roaring Bitmap written to small sibling Parquet file.
   - Indicates deleted row indexes within 1 Parquet file (1 bit per row).
   - Zero file rewrites on DELETE/UPDATE; amortized 10x faster writes!
```

🔍 DELETION VECTOR ENABLEMENT

- **Enable Delta DVs:** `ALTER TABLE tbl SET TBLPROPERTIES ('delta.enableDeletionVectors' = true);`
- **Compaction Trigger:** `OPTIMIZE tbl;` merges deletion vectors back into clean full Parquet files.

⚡ Layout Optimization: Liquid vs Transforms

Clustering

```
-- DELTA LAKE LIQUID CLUSTERING (DBR 13.3+ LTS)
CREATE TABLE gold_events (
  event_id STRING, cust_id BIGINT, event_time TIMESTAMP, payload STRING
) USING DELTA
CLUSTER BY (cust_id, event_time); -- Replaces rigid Hive partitioning & Z-Order!

-- APACHE ICEBERG HIDDEN PARTITIONING & TRANSFORMS
CREATE TABLE iceberg_events (
  event_id string, cust_id bigint, event_time timestamp, payload string
) USING ICEBERG
PARTITIONED BY (
  days(event_time),       -- Hidden transform: no synthetic col required!
  bucket(16, cust_id)     -- Murmur3 hash bucket transform
);
```

🔍 CLUSTERING SUPERIORITY

- **Liquid Clustering:** Non-destructive; keys can be changed anytime via `ALTER TABLE ... CLUSTER BY` without rewriting data.
- **Hidden Partitioning:** Iceberg prevents consumer query errors; `WHERE event_time > '2026-01-01'` automatically prunes day partitions.

🧬 Schema Evolution & Column Mapping

Schema Governance

```
-- 1. Delta Lake Column Mapping (Physical ID vs Logical Name)
ALTER TABLE delta_orders SET TBLPROPERTIES (
  'delta.columnMapping.mode' = 'name',
  'delta.minReaderVersion' = '2',
  'delta.minWriterVersion' = '5'
);
ALTER TABLE delta_orders RENAME COLUMN old_cust_id TO customer_id; -- Zero data rewrite!

-- 2. Iceberg Native Field-ID Architecture
-- Iceberg schemas use immutable Integer field-ids in Parquet metadata.
ALTER TABLE ice_orders RENAME COLUMN cust_id TO customer_id;
ALTER TABLE ice_orders ALTER COLUMN amount TYPE DOUBLE; -- Safe widening (int → long → double)
```

🔍 EVOLUTION INVARIANTS

- **Safe Widening:** Both formats permit widening conversions without file rewrites. Narrowing is forbidden.
- **Column Drop:** Dropping columns marks field as tombstoned in metadata; data is reclaimed during subsequent `OPTIMIZE / VACUUM`.

⏳ Time Travel & Snapshot Auditing

Auditability

```
-- DELTA LAKE TIME TRAVEL
SELECT * FROM silver_orders VERSION AS OF 15;
SELECT * FROM silver_orders TIMESTAMP AS OF '2026-11-10 12:00:00';
RESTORE TABLE silver_orders TO VERSION AS OF 14; -- Instant rollback!

-- APACHE ICEBERG TIME TRAVEL & TAGS
SELECT * FROM ice_orders FOR SYSTEM_VERSION AS OF 102938475892;
SELECT * FROM ice_orders FOR SYSTEM_TIME AS OF '2026-11-10 12:00:00';

-- Iceberg Snapshot Tagging (Releases & Audits):
ALTER TABLE ice_orders CREATE TAG audited_q3_2026 RETAIN 365 DAYS;
SELECT * FROM ice_orders.tag_audited_q3_2026;
```

🔍 POINT-IN-TIME MECHANICS

- **Metadata Pointers:** Time travel reconstructs active files by reading historical snapshot records without moving data files.
- **Retention Dependency:** Time travel is strictly bounded by `VACUUM` or `expire_snapshots` retention policies.

🏛️ DELTA & ICEBERG OPERATIONS & UNIFORM GUIDE

Production Reference: Compaction Runbooks, Delta UniForm, Zero-Copy Migrations & Architectural Decision Matrix

OPERATIONS | DELTA UNIFORM | MIGRATION | ISO 216 A4

🧹 Table Compaction & Maintenance Runbook

FinOps / Hygiene

```
-- DELTA LAKE MAINTENANCE PROTOCOL
-- 1. Compact small files into 128MB-1GB Parquet files
OPTIMIZE delta_tbl WHERE event_date ≥ current_date() - 7;
-- 2. Dry-run safety check before permanently deleting tombstoned files
VACUUM delta_tbl RETAIN 168 HOURS DRY RUN;
VACUUM delta_tbl RETAIN 168 HOURS; -- Default 7-day retention safety threshold

-- APACHE ICEBERG MAINTENANCE (Spark Stored Procedures)
CALL spark_catalog.system.rewrite_data_files(
  table ⇒ 'prod.orders', strategy ⇒ 'binpack',
  options ⇒ map('target-file-size-bytes', '536870912') -- 512MB
);
CALL spark_catalog.system.expire_snapshots(
  table ⇒ 'prod.orders', older_than ⇒ TIMESTAMP '2026-11-01 00:00:00', retain_last ⇒
10
);
CALL spark_catalog.system.remove_orphan_files(table ⇒ 'prod.orders');
```

🔍 RETENTION CAUTION

- **Safety Override:** Never set `spark.databricks.delta.vacuum.parallelDelete.enabled` with zero retention unless restoring corrupted stages.
- **Orphan Removal:** `remove_orphan_files` purges uncommitted staging files left by killed executor tasks.

🌐 Delta UniForm (Universal Format) Deep Dive

Cross-Engine

```
-- Enable Delta Universal Format for Iceberg Clients
CREATE TABLE uniform_telemetry (
  device_id STRING, temp DOUBLE, ts TIMESTAMP
) USING DELTA
TBLPROPERTIES (
  'delta.universalFormat.enabledFormats' = 'iceberg',
  'delta.enableDeletionVectors' = 'true'
);

-- Under the Hood Mechanics:
1. Spark writes native Delta Lake Parquet files & _delta_log/*.json.
2. Async catalog task automatically generates Apache Iceberg metadata:
   - Converts Delta log commit into vN.metadata.json & Avro manifests.
3. Snowflake, AWS Athena, BigQuery, & Presto query identical Parquet files!
-- Zero data duplication, zero dual-write overhead, 100% vector speeds!
```

🔍 QUERYING FROM REMOTE ENGINES

- **Snowflake External Table:** Point Snowflake to Iceberg metadata location generated by UniForm; full read-performance.
- **UniForm Supported Features:** Supports Deletion Vectors (translated to Iceberg positional deletes) and column mapping.

🔄 In-Place Zero-Copy Migration Blueprint

Migration

```
-- 1. Migrate Parquet Table to Delta Lake in-place (Zero copy!)
CONVERT TO DELTA parquet.`s3://prod-lake/legacy_orders/`
PARTITIONED BY (order_year INT, order_month INT);

-- 2. Migrate Parquet Table to Apache Iceberg via Stored Procedure
CALL spark_catalog.system.migrate(table ⇒ 'hive_prod.legacy_orders');
-- Or Non-destructive Snapshot migration (keeps original table intact):
CALL spark_catalog.system.snapshot(
  source_table ⇒ 'hive_prod.legacy_orders', table ⇒ 'iceberg_prod.orders_ice'
);

-- 3. Ingesting Loose External Parquet Files into Iceberg Table
CALL spark_catalog.system.add_files(
  table ⇒ 'iceberg_prod.orders_ice',
  dir ⇒ 's3://external-bucket/incoming_parquet/'
);
```

🔍 MIGRATION INVARIANTS

- **Parquet Compatibility:** Existing Parquet files must have consistent column types; no data re-write occurs during `CONVERT`.
- **Rollback Protocol:** Snapshot migration generates an isolated metadata branch, preserving the original Hive table untouched.

📡 Streaming I/O & Change Data Feed (CDF)

Streaming CDC

```
-- 1. Delta Lake Change Data Feed (CDF)
ALTER TABLE silver_customers SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');

-- Read CDC stream in PySpark:
cdc_stream = spark.readStream.format("delta")   .option("readChangeFeed", "true")   .op
tion("startingVersion", 10)   .table("silver_customers")
-- Emits: _change_type ('insert', 'update_preimage', 'update_postimage', 'delete')

-- 2. Apache Iceberg Incremental & CDC Reads
ice_stream = spark.readStream.format("iceberg")   .option("streaming-skip-delete-marker
s", "true")   .load("prod.customers")
```

🔍 DOWNSTREAM PROPAGATION

- **Silver-to-Gold Pipeline:** Consume `update_postimage` to maintain synchronized downstream materialized views.
- **Watermarking:** Delta CDF supports stateful structured streaming watermarks on commit timestamp (`_commit_timestamp`).

🏛️ Multi-Engine Catalog Integration

Ecosystem

```
-- Lakehouse Open Catalog Architectures
1. Unity Catalog (Delta & UniForm Iceberg):
   - Centralized RBAC across Databricks, external Spark, & UniForm clients.
   - Native column-level masking, row-level filters, and lineage tracking.
2. Apache Iceberg REST Catalog Standard:
   - Open OpenAPI HTTP specification for table snapshot transactions.
   - Implementations: Apache Polaris, Project Nessie, AWS Glue, Tabular.
   - PyIceberg (Python native client) connects without Java/JVM runtime!
3. Cross-Cloud Engine Query Compatibility:
   - Snowflake: Reads Iceberg via REST Catalog; reads Delta via UniForm.
   - Google BigQuery: BigLake native integration for Iceberg & Delta.
   - AWS Athena / EMR: Native read/write on both formats via AWS Glue.
```

🔍 CATALOG PROTOCOL

- **Vendor Neutrality:** Iceberg REST catalog protocol prevents cloud vendor lock-in with standard HTTP commit calls.
- **Pyiceberg:** Enables lightweight AWS Lambda / Cloud Functions to execute ACID queries without starting Spark clusters.

⚖️ Head-to-Head Architectural Decision Matrix

Strategy

-- Architectural Evaluation: Delta Lake vs Apache Iceberg

Capability	Delta Lake 3.2+	Apache Iceberg 1.5+
Primary Engine	Spark / Photon Vector	Multi-Engine (Trino/Flin)
Log Structure	Linear JSON + Checkpoint	DAG of Avro Manifests
Row Mutations	Deletion Vectors (Bitmaps	Positional/Equality Avro
Layout Optimization	Liquid Clustering	Hidden Partitions + Sort
Partition Evolution	Rewrite required	Native In-Place Metadata
Catalog Requirement	Optional on cloud storage	Mandatory (REST/Glue)
Multi-Engine Interop	UniForm (generates Ice)	Native across all engines
Python Client	delta-rs (Rust/Python)	PyIceberg (Pure Python)

🔍 SELECTION RECOMMENDATION

- **Choose Delta Lake:** If primary compute is Databricks/Spark and workloads demand highest single-engine Photon throughput.
- **Choose Iceberg:** If architecture spans heterogeneous engines (Trino, Flink, Snowflake, Spark) with dynamic partition evolution.